

IA Aplicada com LLMs

Aula 06 — O router por embedding

Onde paramos

A aula entregou um buscador: indexa um regulamento e devolve o artigo que responde a pergunta.

Falta uma dívida, e ela foi contraída há duas semanas.

A Aula 05 deixou isto escrito

Forma	Como decide	Preço
Regra	palavra-chave, regex, metadado	zero chamadas
Embedding	vetor da entrada × exemplos de rota	é a Aula 06
Semântico	um modelo devolve {"rota": "..."} 	uma chamada de geração

E o laboratório de lá levou a dívida ao código:

```
router_embedding = None    # Aula 06
```

É aqui.

Roteiro

1. Classificar é buscar, com outro índice
2. A margem importa mais que o score
3. Onde ele quebra
4. A cascata, completa pela primeira vez
5. O limiar, e a rota que diz "não sei"

Objetivos

- **Implementar** um classificador por similaridade, sem treino e sem geração
- **Reconhecer** que buscar e classificar são a mesma operação
- **Comparar** as três formas de rotear por custo e condição de não uso
- **Diagnosticar** decisão frágil pela **margem**, não pelo score
- **Justificar** a ordem da cascata pelo limite conhecido de cada degrau

Parte 1

Classificar é buscar

O que muda é só o que está no índice

BUSCADOR

indexa: chunks do documento
devolve: os k mais próximos

ROUTER

indexa: frases-exemplo de cada rota
devolve: a rota do mais próximo

```
EXEMPLARES = {  
    "consulta_status": ["qual o status do meu pedido",  
                        "onde está minha encomenda", ...],  
    "reclamacao":      ["meu pedido está atrasado e ninguém resolve", ...],  
    "fora_de_escopo":  ["vocês têm vaga de emprego", ...],  
}
```

Sem treino, sem *prompt*, sem geração. Acrescentar uma rota é acrescentar uma **chave no dicionário**.

O router é o **ranquear** do script 01

A assimetria do script 04 vale igual:

```
embutir os exemplares ... UMA VEZ, na largada  
classificar uma mensagem 1 chamada – a dela
```

Nenhum mecanismo novo nesta nota. O que o aluno escreveu para buscar documentos classifica mensagens sem uma linha a mais.

A margem importa mais que o score

```
consulta_status  score 0.8482  margem +0.1320  <- decisão folgada  
consulta_status  score 0.7761  margem +0.0108  <- empate, e ERRADO
```

Mesma rota, scores da mesma ordem. Só a segunda está errada: é uma reclamação sobre caixa amassada.

Margem = distância para o exemplar mais próximo de **outra** rota.

O score não denuncia. A margem sim — a 0,0108, o segundo lugar ficou a um centésimo, e o sistema não sabe que quase decidiu o contrário.

Margem estreita é candidata a **revisão humana**, não a decisão automática.

Parte 2

Onde ele quebra

Uma palavra de diferença

"o pedido 48219 chegou certinho, obrigado"	-> agradecimento
"o pedido 48219 não chegou"	-> entrega perdida

consulta_status	0.8313	margem +0.0638	"chegou certinho, obrigado"
consulta_status	0.7963	margem +0.0041	"não chegou"
cosseno entre as duas 0.8908			

A mesma rota para os dois.

E repare no contexto

sem corpus
sem trechos recuperados
sem nenhum modelo escrevendo uma palavra

Não é um problema de RAG. É um problema de **representação**, e aparece em qualquer sistema que decida por proximidade — inclusive num classificador de três rotas.

A Aula 07 volta a este caso, e mostra que ele não é isolado.

A saída é a cascata, e agora ela tem três degraus

Degrau	Custo	Resolve	Não resolve
Regra	zero	polaridade, número, id, data	redação não enumerada
Embedding	1 chamada barata	a mesma coisa, com outras palavras	o que o vetor não vê
Semântico	1 chamada de geração	o que ninguém previu	o custo, em volume

```
def regra(mensagem: str) -> str | None:
    baixo = mensagem.lower()
    if any(t in baixo for t in ("não chegou", "nao chegou", "não recebi")):
        return "reclamacao"
    return None
```

Cada degrau existe por um limite conhecido

REGRA	->	não generaliza
EMBEDDING	->	não vê polaridade, magnitude, identidade, ordem
SEMÂNTICO	->	custa uma geração por item

Um router que **começa** no degrau mais caro não economiza nada.

Um router que **nunca sobe** erra nos casos que não previu.

O limiar, e a rota que diz "não sei"

```
piso 0.00 -> consulta_status    score 0.7718  
piso 0.82 -> nenhuma           nada acima do piso
```

Sem piso, o classificador **sempre** devolve uma rota — a menos ruim.

Mesmo defeito que a Aula 05 apontou no router semântico: sem escape, ele é obrigado a escolher, e escolhe.

A rota `nenhuma` reaparece na Aula 07 com outro nome: `suficiente: false`.

E o piso não se chuta

Calibra-se contra um conjunto de mensagens com **rota conhecida** — do mesmo jeito que o `k` se escolheu no script 02.

É o conjunto de avaliação desta aula, em outro uso.

A nota 01 §5 já tinha avisado: o piso real entre textos sem relação nenhuma fica **bem acima de zero**. Um limiar absoluto no chute não funciona.

O que fica

- Buscar e classificar são a **mesma operação** — muda o que se indexa
- O router por embedding decide **sem gerar texto**
- A **margem** diz mais que o score
- O erro de rota acontece **sem nenhum modelo escrever uma palavra**
- A cascata tem três degraus, e cada um tem limite conhecido

A dívida da Aula 05 está paga. Próxima nota: os vetores saem da memória e vão para um **banco** — e a aula mede quanto isso custa.